

Trabajo Práctico 02

Laboratorio de Datos

Segundo Cuatrimestre - 2026

Integrantes: Aviles Clara, Cabrera Maria Rosario, Roux Vera

Grupo: GrupoCRV

Comisión: A

Fecha de entrega: 01/11/2026

RESUMEN

En el siguiente informe vamos a explicar paso a paso los diferentes experimentos que realizamos con modelos de clasificación a partir del dataset de Quick, Draw! Presentaremos los resultados relevantes junto con una explicación de los mismos, acompañada de gráficos que los complementen. En esta primera parte, entrenamos modelos de K-Nearest Neighbors (KNN) para clasificar únicamente entre las clases Cuadrado y Casa, observamos cómo variaba la medida de accuracy cuando modificamos las distintas variables: K (de 1 a 100), la elección de atributos para los subconjuntos y su tamaño (de 3 a 360). Para elegir los atributos, usamos como criterios los píxeles de mayor brillo promedio por clase y los de mayor diferencia de intensidad entre clases.
En la segunda parte de este trabajo utilizamos distintas variables para armar árboles de decisión para luego clasificar en las distintas clases del dataset. Variamos la altura del árbol entre 1 y 10 y el criterio de división entre gini y entropy, y evaluamos cada combinación con cross validation sobre el conjunto de desarrollo. Complementamos la métrica con una matriz de confusión, que nos permitió ver entre qué clases el modelo se equivoca más. Por último, a lo largo del informe analizamos el overfitting y el balance entre sesgo y varianza, tanto al variar el K en KNN como al aumentar la profundidad de los árboles.

INTRODUCCIÓN

Comenzamos haciendo un análisis exploratorio del dataset para poder decidir cuáles eran los datos más relevantes que nos serían de utilidad para los dos problemas que tendríamos que resolver a continuación: primero, entrenar un modelo con KNN que clasifique entre la clase 2 (Cuadrado) y la 9 (Casa) y luego un árbol de decisión que pudiera clasificar entre las 10 clases de nuestro dataset.

Previamente revisamos que la cantidad de muestras por clase se encuentre balanceada. Esto nos va a servir a lo largo del análisis para decidir las métricas en ambos experimentos. Lo graficamos para mejor entendimiento (Ver Gráfico 1: Cantidad de muestras por clase)

Para empezar, tomamos en cuenta dos criterios que pensamos que combinados pueden ser útiles para reducir la cantidad de atributos que tenía cada clase.
Por un lado, tomamos los píxeles que más brillan en promedio entre todas las imágenes de cada clase. Aunque estos resultaron no ser tan representativos porque hay puntos más intensos compartidos entre muchas clases. En concreto, las clases 3, 5 y 8 coinciden en muchos píxeles cuando vemos cuáles tienen mayor intensidad en promedio. Lo mismo pasa para las clases 0, 2, 5 y 7 que coinciden en las “partes de arriba” o “los techos”, por ejemplo, el píxel 42. La clase triángulo tiene algunos puntos fuertes que no coinciden con ningún otro, posiblemente las esquinas inferiores. Lo observamos en el Gráfico 2 que muestra todas las imágenes promedio de cada clase.

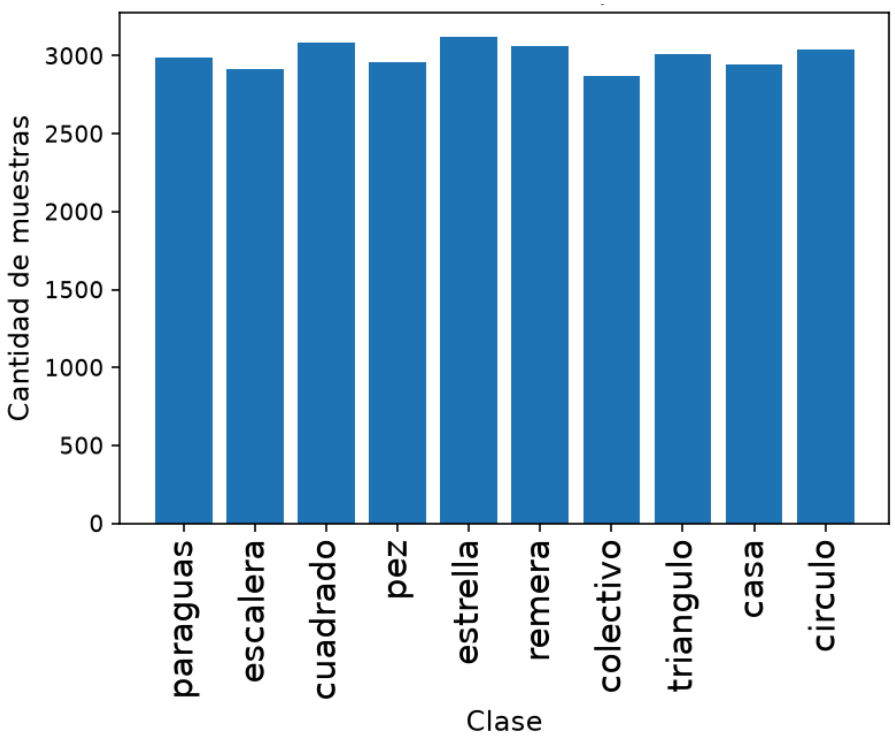


Gráfico 1: Cantidad de muestras por clase

Por esto también usamos el criterio de la mayor diferencia de intensidad de píxeles. Calculamos el brillo promedio de cada píxel por clase y le tomamos el valor absoluto a la resta píxel a píxel entre clases. Los píxeles que quedaron con números más altos representan aquellos que brillan mucho en una figura y poco en otra, por lo que son útiles para poder diferenciarla.

También creemos que se pueden descartar atributos. Hay muchos píxeles que tienen luminosidad cero ya que las figuras generalmente no están rellenadas, por lo que todos los píxeles que son el “fondo” se pueden descartar. Lo mismo sucede con las esquinas; la mayoría no están pintadas.

Por ejemplo, si quisiéramos diferenciar entre un círculo y un paraguas, habría que mirar el interior del círculo ya que este tiene 0 de intensidad y ahí se encuentra todo el cuerpo del paraguas (ver Gráfico 3: Diferencia entre círculo y paraguas). Pero entre el cuadrado y el círculo es mejor ver las esquinas, ya que por la curvatura del círculo ahí no coinciden.
Otro ejemplo claro es si pensamos en el colectivo y el pez. Al mirar las figuras promedio, nos damos cuenta de que ocupan aproximadamente los mismos píxeles; sin embargo, el colectivo tiene una línea superior mucho más brillante que podría ayudar a distinguirlas (ver Gráfico 4: Diferencia entre colectivo y pez). Ahora, esos píxeles en particular serían útiles para diferenciar entre estas dos clases, pero si lo comparamos con el cuadrado, esa línea más brillante pasa a ser compartida y ahí deberíamos tomar otros atributos.

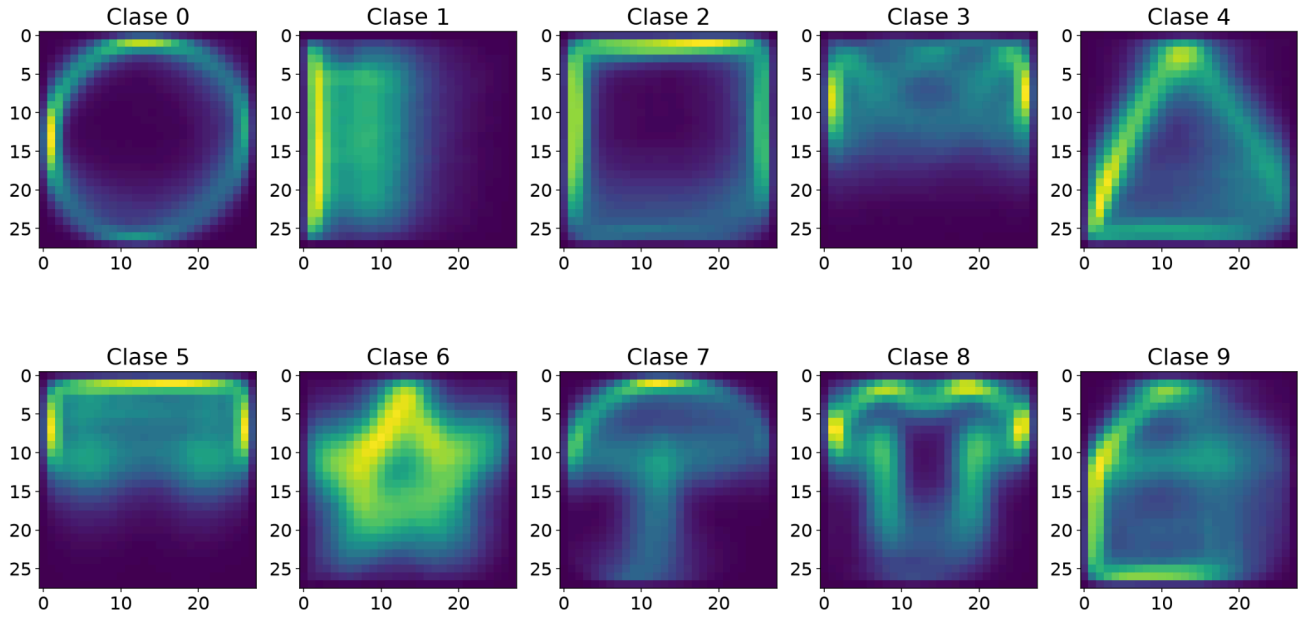


Gráfico 2: Dibujo promedio por clase

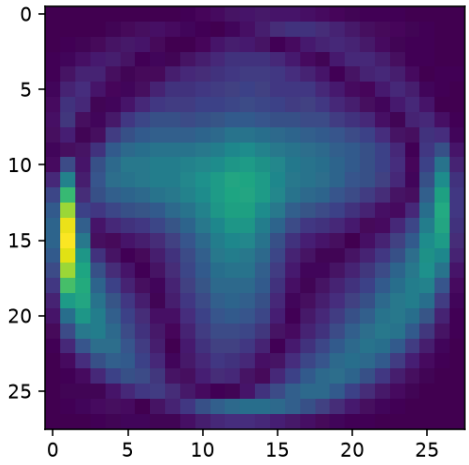


Gráfico 3: Diferencia entre círculo y paraguas

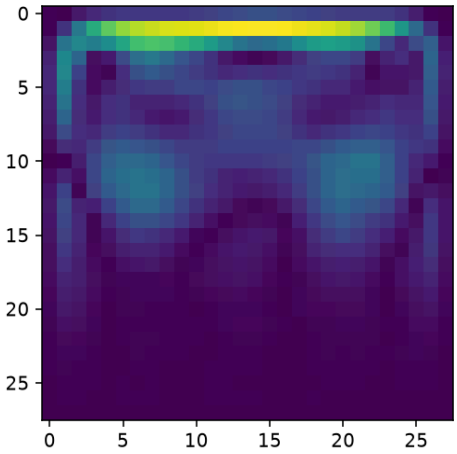


Gráfico 4: Diferencia entre colectivo y pez

Podemos usar esta información para responder la pregunta planteada en la primera parte: ¿Hay objetos cuyos dibujos son más parecidos entre sí? Por ejemplo, ¿son más fáciles de diferenciar las imágenes correspondientes a la clase 2 (cuadrado) de las de la clase 9 (casa) que de las de la clase 6 (estrella)? Consideramos que efectivamente sucede que hay clases con dibujos más parecidos entre sí. Primeramente, esto lo podemos ver en el Gráfico 2: Dibujo promedio por clase. Si observamos el cuadrado, podemos diferenciarlo casi perfectamente; los trazos están bastante marcados. Lo mismo podemos ver en el dibujo de la estrella. La casa, por otro lado, se difumina más al costado, lo que significa que los promedios dieron más bajos en esos píxeles y que varía más el trazo en ese lado. Queremos agregar también aquí a la clase 1 (escalera), cuya imagen promedio no es fácilmente identificable; sólo brilla claramente la línea izquierda. Esto se comprende mejor si miramos el data frame que contiene los 10

puntos más brillantes promedio por clase (pixeles_mas_intensos). Vemos que la clase cuadrado tiene sus puntos brillantes promedio mucho más intensos, con valores desde 188 hasta 221, es decir, son puntos representativos que se deben compartir en la gran mayoría de los cuadrados. Luego, si miramos la intensidad de los puntos de la escalera o la casa, vemos números que rondan los 135 a 150, lo cual nos dice que a pesar de tener puntos en común, estos dibujos no son tan parecidos como los cuadrados.

Esta fue la misma lógica que aplicamos para responder si todas las imágenes de la clase 3 (pez) son muy similares entre sí: justamente la clase 3 es la que tiene mayor variabilidad en sus diseños. Lo observamos rápidamente en el Gráfico 2, donde no se puede distinguir la silueta del pez. Si miramos con más atención, se pueden ver dos peces superpuestos, mirando hacia los lados contrarios. Al ser una figura poco simétrica (es más, pareciera ser la única figura no simétrica de la lista), varía mucho si se encuentra dibujada para un lado o para el otro. Seguramente se pueda separar en dos grandes grupos, los peces que tienen la cabeza a la izquierda y los que la tienen a la derecha, y entre esos, las subclasificaciones serán mucho más parecidas entre sí.
Si miramos los píxeles de mayor intensidad promedio del pez, primeramente vemos que no brillan tanto como los del cuadrado, alcanzando el valor máximo de 170 y con bastante variabilidad, comenzando en 145. Creemos que los más brillantes son los que se superponen en las dos formas de dibujar al pez; por ejemplo, debe haber píxeles que sean tanto un pedazo de la curvatura de la cabeza como de la cola, que suele tener forma de triángulo.

En cuanto a la última pregunta de la primera parte, consideramos que a partir de este dataset compuesto por imágenes la exploración resulta un poco más compleja que cuando utilizabamos datasets como el del Titanic, visto en clase. Esto es porque cada imagen consta de 784 píxeles, que corresponden a 784 atributos separados, lo cual resulta difícil entender qué representa cada uno por su cuenta. Por este motivo, para el análisis exploratorio tuvimos que pensar cómo visualizar ejemplos individuales y calcular imágenes promedio de cada clase, lo que nos permitió entender mejor el dataset.

EXPERIMENTOS REALIZADOS

CLASIFICACIÓN BINARIA

Para este primer experimento vamos a utilizar el modelo de clasificación de K-Nearest Neighbors para entrenar un modelo que pueda predecir entre las clases 2 (Cuadrado) y 9 (Casa).

Como vimos en el análisis exploratorio de nuestro dataset, las clases se encuentran balanceadas. En particular, las dos que nos interesan ahora se encuentran formadas por 3085 muestras en la clase 2 y 2945 en la clase 9. Esto, sumado a que todas las clases tienen la misma relevancia dentro del problema y no existe un tipo de error de clasificación que deba ser priorizado o penalizado más que otro, nos lleva a elegir la métrica Accuracy, la cual permite medir el rendimiento global del modelo, considerando tanto los falsos positivos como los falsos negativos.

El objetivo de esta parte es experimentar con diferentes combinaciones de parámetros e hiperparámetros e ir reportando cómo varía la métrica con ellos. Para esto comenzamos dejando un K fijo en 5 de manera arbitraria, ya que el análisis del K lo hacemos luego de obtener los atributos. Además, armamos 4 subgrupos de parámetros utilizando los criterios que mencionamos en el análisis exploratorio para definir qué cantidad de atributos íbamos a utilizar para entrenar modelos en donde varíe el hiperparámetro de cantidad de vecinos. Los dos primeros subconjuntos correspondían a los píxeles más brillantes del Cuadrado y de la Casa respectivamente; luego, los píxeles más intensos de la diferencia entre ambas clases y finalmente una combinación entre los primeros tres. Asimismo, fuimos variando los tamaños de los subconjuntos: creamos una función para automatizar el armado de los mismos y poder construir subconjuntos de los tamaños que deseamos. Para esta primera parte utilizamos subconjuntos de tamaños desde 3 hasta 360, con saltos de 9. Al principio, la métrica va mejorando a medida que aumenta la cantidad de píxeles. Creemos que, en parte, es porque los píxeles que más (o menos) brillan suceden por zonas (por ejemplo, los píxeles del 40 al 50 en el caso del cuadrado), por lo que aumentar la cantidad del subconjunto hace que aparezcan puntos de otras zonas y el modelo pueda clasificar mejor. De todas formas, el crecimiento no es constante, de hecho, se alcanza la mejor accuracy con 0.9834 en el subconjunto de 255 atributos de “Diferencias” y no en el de 360 (Ver Gráfico 5: Variación del ACC según cantidad de atributos para los distintos subconjuntos, k=5).

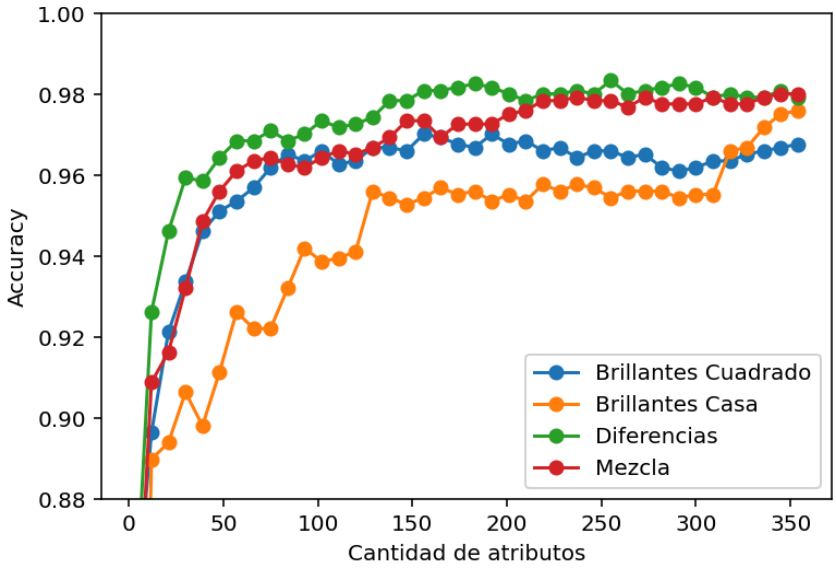


Gráfico 5: Variación del ACC según cantidad de atributos para los distintos subconjuntos, k=5

Observamos en el Gráfico 5 que, dentro de cada cantidad, el conjunto que mejor mide es la diferencia (en el archivo Python se encuentra también el gráfico con la escala para que se vea la totalidad de los datos; decidimos incluir este en el informe ya que se aprecia mejor el alcance del valor máximo). Asimismo, la mayoría de los píxeles que tienen mayor diferencia se encuentran entre los más brillantes de la clase 2 (por lo que también deben ser de los menos brillantes en la clase 9). Creemos que esto se debe a que, a diferencia del subconjunto Mezcla, que se arma de manera arbitraria; y los subconjuntos de píxeles más brillantes de cada clase

analizada, que se arman con información de cada clase respectivamente; el elegido tiene información relevante de ambas clases.

Decidimos quedarnos con los subconjuntos de 255 atributos y utilizarlos para entrenar diferentes modelos, variando K de 1 a 100, con saltos de 5, para ver qué combinación maximiza la métrica. Previamente probamos distintos máximos y notamos que luego de k = 100 el accuracy continúa bajando sin variar significativamente. Es por esto que decidimos graficar con este límite.

A partir de los resultados obtenidos, pudimos observar que la mejor accuracy por K es en su mayoría la que corresponde al grupo de atributos “diferencias” (Ver Gráfico 7: Variación del ACC según K para diferentes combinaciones de atributos). Específicamente, la mejor combinación resultante dentro de los parámetros que probamos es el modelo con K = 1 de 255 atributos correspondientes a los píxeles de mayor diferencia de brillo entre las clases. Además, en el Gráfico 7 también notamos que la mejoría no está directamente relacionada con el aumento de K.

(Ver Gráfico 6: Variación del ACC según K por Train y Test para el subconjunto “Diferencias” con 255 atributos)

En el Gráfico 6 no solo se muestra claramente que el conjunto de test alcanza su valor máximo de accuracy en K = 1, sino que también evidencia el comportamiento de este tipo de modelos. Cuando el K es pequeño, el modelo es más sensible a las particularidades de los datos de entrenamiento y puede presentar mayor varianza y sobreajuste. En cambio, con K grande el modelo realiza predicciones más generales, lo que reduce la varianza, pero puede aumentar el sesgo y provocar subajuste. En KNN, cuando evaluamos el modelo sobre los datos de entrenamiento, cada observación puede encontrarse a sí misma entre sus vecinos más cercanos; es por esto que en las K más pequeñas el conjunto de Train obtuvo valores más elevados.

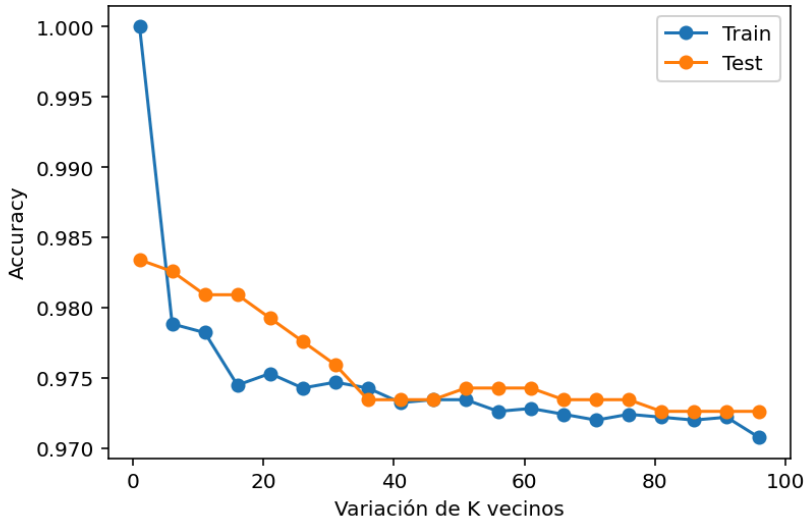


Gráfico 6: Variación del ACC según K por Train y Test para el subconjunto “Diferencias” con 255 atributos

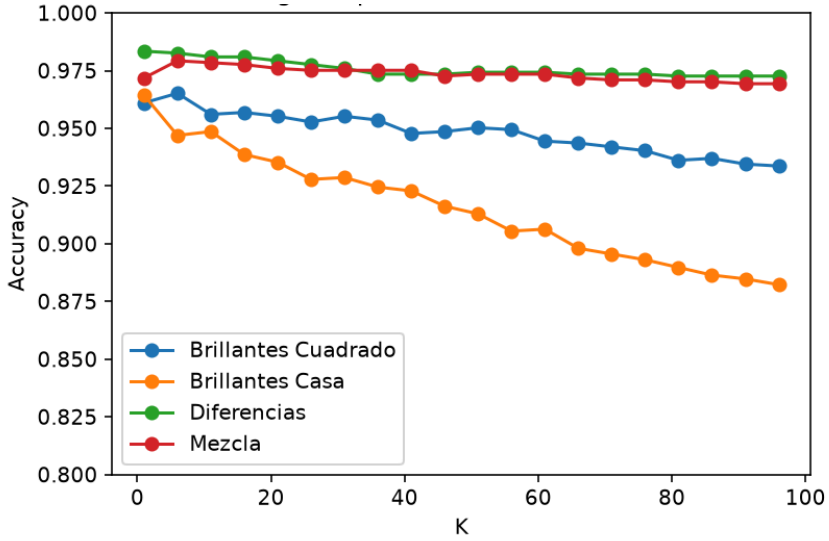


Gráfico 7: Variación del ACC según K para diferentes combinaciones de atributos

Mirando el conjunto Test (Ver Gráfico 6: Variación del ACC según K por Train y Test para el subconjunto “Diferencias” con 255 atributos), notamos que para este la K = 1 también es el mejor valor. Creemos que esto se debe al tipo y la cantidad de atributos elegidos, los cuales son representativos de cada clase y permiten separar bien los datos. De hecho, si miramos un conjunto con menor cantidad de atributos, como por ejemplo 9, observamos que no sucede lo mismo que con 255. En este caso, alcanza su máxima accuracy de 0.9245 en K = 16. (Ver Gráfico 8: Variación del ACC según K por Train y Test para el subconjunto “Diferencias” con 9 atributos)

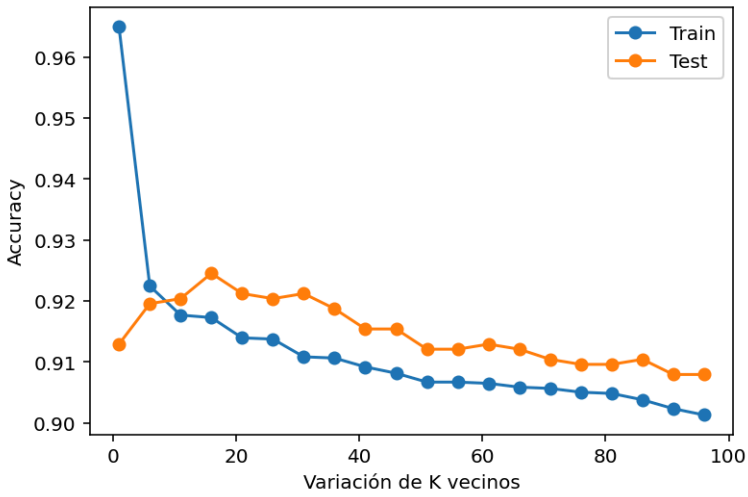


Gráfico 8: Variación del ACC según K por Train y Test para el subconjunto “Diferencias” con 9 atributos

CLASIFICACIÓN MULTICLASE

El segundo experimento consiste en encontrar la mejor combinación de hiperparámetros para entrenar un modelo de árbol de decisión que clasifique entre las 10 clases del dataset.
Comenzamos separando el conjunto en desarrollo y held out. En esta ocasión utilizaremos cross validation para entrenar y calcular la métrica con el conjunto de desarrollo. Con el held out podremos estimar qué tan bien va a predecir nuestro modelo sobre datos que jamás vio. Utilizaremos nuevamente la métrica de accuracy, por las mismas razones mencionadas en el inciso anterior, y complementaremos con la matriz de confusión.

Los dos hiperparámetros a variar son la altura del árbol y el criterio (gini y entropy), por lo que entrenamos un modelo por cada combinación entre ellos, con alturas entre 1 y 10 como pide la consigna. De todas formas, creemos que vale la pena mencionar que si fuésemos un poco más lejos hasta alturas de 15, llegaríamos a un árbol que tiene un nodo por dato (2 elevado a la 15 supera la cantidad de imágenes de nuestro dataset), lo cual es un claro ejemplo del overfitting. Lo exponemos en el Gráfico 9, donde se observa que las curvas del conjunto train alcanzan una accuracy de 1.

De todas formas, decidimos quedarnos con la altura de valor 10 ya que fue la que mejor midió dentro del rango pedido por consigna. Nuestro mejor modelo es el que tiene dicha altura junto con el criterio de entropía (Ver Gráfico 9: Variación del ACC según diferentes profundidades de árbol y distintos criterios por Train y Test). Dentro de este rango, las curvas de test alcanzan un valor máximo de accuracy de 0.7804. Aquí podemos ver más en profundidad cómo predice nuestro modelo.

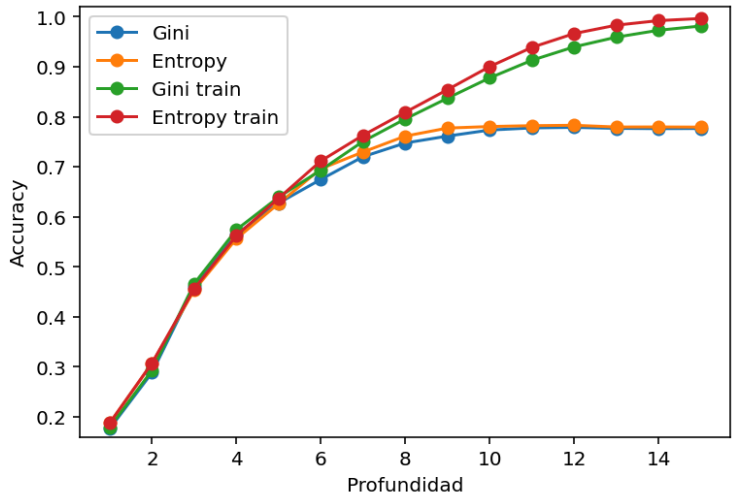
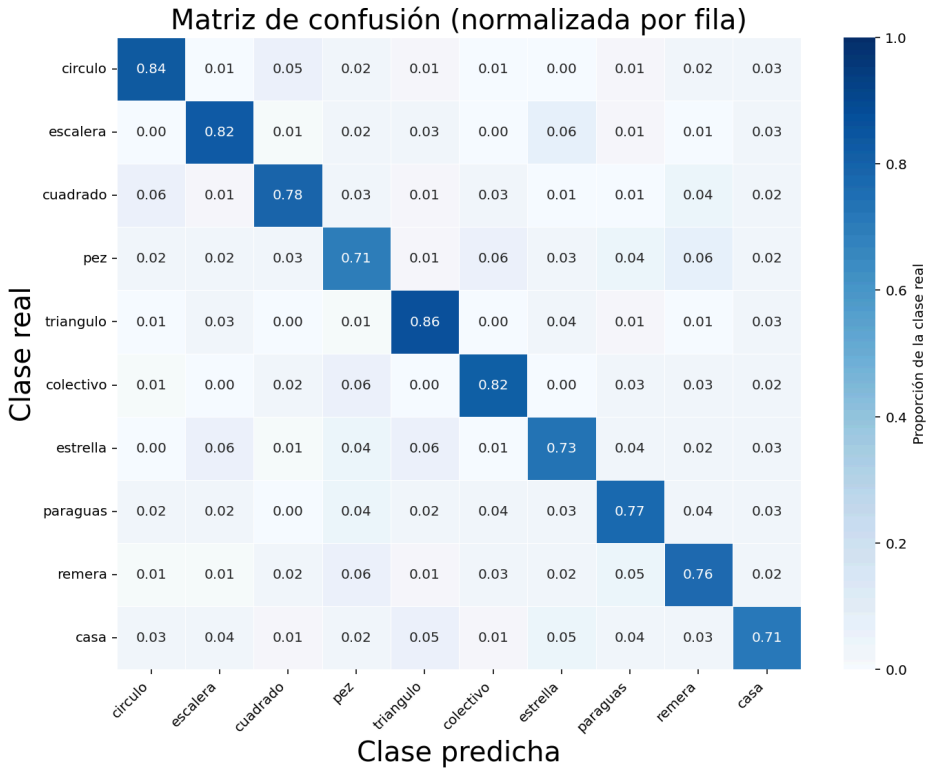


Gráfico 9: Variación del ACC según diferentes profundidades de árbol y distintos criterios por Train y Test

También adjuntamos una tabla de confusión normalizada, en forma de heatmap, la cual nos muestra, por ejemplo, que la clase triángulo es la que predice con mayor precisión (0.86), mientras que en clases como pez, casa y estrella son las que tienen más error (valores de 0.71 y 0.73). Estos resultados concuerdan con lo estimado en el análisis exploratorio, donde efectivamente afirmamos que hay imágenes de clases que son más fáciles de diferenciar.



Con esta información entrenamos a nuestro mejor árbol con la totalidad del conjunto de desarrollo e hicimos las predicciones con el held out, obteniendo una accuracy de 0.7988. Observamos que es un poco mayor que la que habíamos obtenido sobre el conjunto de desarrollo, ya que el modelo final fue entrenado con el conjunto total y no con los folds.

La accuracy que obtuvimos en esta última parte del análisis resulta de calcular la métrica sobre un nuevo conjunto de imágenes que nuestro modelo no conocía. Esto mismo debería suceder cuando se lo utilice con el conjunto de evaluación de la cátedra, por lo que creemos que la accuracy va a ser también de 0.7988 como la del inciso d y un poco superior a la del inciso c, ya que ahora el modelo fue entrenado con más datos.

CONCLUSIONES

A partir del análisis exploratorio observamos que las clases están balanceadas, por lo que accuracy resultó una métrica adecuada en ambos experimentos. Con las imágenes promedio y la comparación del brillo por clase vimos que algunas clases son más fáciles de distinguir que otras.

Del primer experimento podemos concluir que el subconjunto de atributos más representativo para distinguir entre las clases Cuadrado y Casa es el de las diferencias, ya que este permite encontrar los píxeles en los que cada clase se diferenciaba del resto; en cambio, en los subconjuntos de los más brillantes había píxeles que se repetían para diferentes clases. Luego, al evaluar con diferentes cantidades de atributos, la mejor accuracy se dio con 255, no con el

máximo evaluado, por lo que concluimos que la accuracy no necesariamente mejora con un mayor tamaño.
Vimos que si los atributos son los más relevantes para la clasificación, a mayor cantidad, la métrica mide mejor a menor K. Lo comprobamos cuando comparamos el mismo tipo de subconjuntos (diferencias) pero con diferentes cantidades de píxeles. Finalmente, pudimos observar que el conjunto de train siempre mide mejor en K pequeños ya que se encuentra a sí mismo dentro de los vecinos.

Del segundo experimento pudimos concluir que el mejor árbol posible variando ambos hiperparámetros era aquel de altura 10 y criterio de entropía. También pudimos ver más en detalle con la matriz de confusión que hay clases que son más similares entre sí y entre cuáles clases es más difícil distinguirlas.
El modelo final alcanza una accuracy de 0.7988 cuando lo entrenamos con todo el conjunto de desarrollo y lo evaluamos en el held out y debería tener un rendimiento similar cuando se clasifiquen las imágenes del conjunto de evaluación de la cátedra.

BIBLIOGRAFÍA

[1] Ha, David, and Douglas Eck. "A Neural Representation of Sketch Drawings." arXiv preprint arXiv:1704.03477 (2017).